

# MODULE 6

## ALIGNMENT, GOVERNANCE & ORGANISATIONAL TRANSFORMATION

Professional Certificate in Machine Learning — Terrene Open Academy

*"Your competitors are scrambling. You'll be ready."*

## WHERE WE'VE BEEN

| Module | What You Learned                    | Kailash Engines                     |
|--------|-------------------------------------|-------------------------------------|
| M1     | Python, Polars, data exploration    | DataExplorer, PreprocessingPipeline |
| M2     | Statistics, experiments, features   | FeatureStore, ExperimentTracker     |
| M3     | Supervised ML, production pipelines | TrainingPipeline, ModelRegistry     |
| M4     | Unsupervised, NLP, deep learning    | AutoMLEngine, DriftMonitor          |
| M5     | LLMs, agents, RAG, deployment       | Delegate, ReActAgent, Nexus         |

## THE FINAL FRONTIER

---

Module 6 answers three questions no other module touched:

1. **How do we teach models our values?** — Alignment (6.1–6.4)
2. **Who controls the AI?** — Governance (6.5–6.7)
3. **How does an agent learn from experience?** — Reinforcement Learning (6.3)

**New engines today:** AlignmentPipeline, AdapterRegistry, GovernanceEngine, PactGovernedAgent, RLTrainer

## TODAY'S ROADMAP

---

6.1 SFT/LoRA → 6.2 DPO → 6.3 RL → 6.4 Merging → 6.5 PACT → 6.6 Gov. Agents → 6.7 Scale → 6.8 Capstone

Alignment block first (teaching the model), then governance block (controlling the organisation).

## OPENING CASE: THE EU AI ACT

---

### First fines expected 2025. Up to 7% of global turnover.

- The EU AI Act entered into force August 2024
- Article 5 (prohibited practices) enforceable from February 2025
- Full compliance for high-risk systems by August 2026
- Singapore AI Verify released ISAGO 2.0 — voluntary but market-expected

**The hook:** Governance is not compliance theatre — it is competitive advantage. Organisations that build governance in from day one ship faster than those retrofitting it.

## WHY THIS MODULE MATTERS

---

### WITHOUT GOVERNANCE

- Agent burns \$50k in API calls overnight
- Model leaks PII from training data
- Intern deploys unreviewed model to production
- No audit trail when regulator asks "why did it decide this?"

### WITH GOVERNANCE

- Cost budgets enforced per-agent, per-task
- Differential privacy baked into training
- Role-based access: only senior ML can promote to prod
- Every decision traceable through PACT audit chain

## REAL CONSEQUENCES

| Incident           | Cause                  | Cost                                       |
|--------------------|------------------------|--------------------------------------------|
| Air Canada chatbot | No operating envelope  | Ordered to honour fabricated refund policy |
| Samsung code leak  | No data classification | Proprietary code sent to ChatGPT           |
| Italy ChatGPT ban  | GDPR non-compliance    | Temporary market exit                      |
| NYC hiring tool    | No bias audit          | \$25K+ fines per violation                 |

Every incident above is preventable with the tools you will learn today.

## 6.1 — SUPERVISED FINE-TUNING

---

### WHAT IS FINE-TUNING?

A pre-trained model already "knows" language. Fine-tuning is like giving a new employee **on-the-job training** — they already speak the language, but need to learn your company's way of doing things.

#### PRE-TRAINED

Knows grammar, facts, reasoning

#### FINE-TUNED

Knows *your* tone, *your* domain, *your* format

## FULL FINE-TUNING VS LORA

---

### FULL FINE-TUNING

- Update ALL parameters
- 7B model = 28 GB of gradients
- Need multiple A100 GPUs
- Catastrophic forgetting risk

### LORA

- Freeze the original weights
- Add tiny trainable matrices
- 0.1–1% of parameters
- Single GPU, laptop-viable

**Analogy:** Full fine-tuning rewrites the entire textbook. LoRA adds sticky notes to the pages that matter.

## LORA: THE CORE IDEA

---

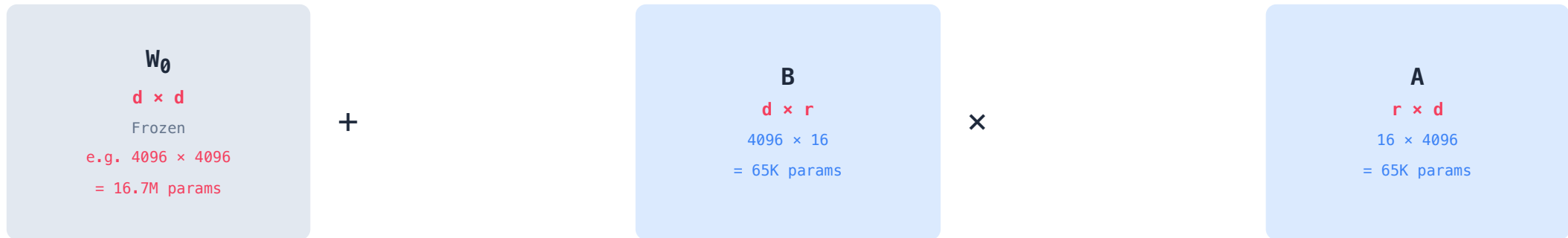
In a transformer, each attention layer has a weight matrix  $W_0 \in \mathbb{R}^{d \times d}$ . LoRA decomposes the update as a **low-rank product**:

$$W = W_0 + \Delta W = W_0 + BA$$

where  $B \in \mathbb{R}^{d \times r}$ ,  $A \in \mathbb{R}^{r \times d}$ , and  $r \ll d$

Only  $B$  and  $A$  are trained.  $W_0$  stays frozen.

## LORA: MATRIX DIMENSIONS



Trainable: 130K vs 16.7M – **0.78% of original parameters**

## WHY DOES LOW RANK WORK?

---

Aghajanyan et al. (2020) showed that pre-trained models have a **low intrinsic dimensionality**:

- The weight updates during fine-tuning live in a low-dimensional subspace
- Most of  $\Delta W$ 's singular values are near zero
- Rank  $r = 8$  to  $r = 64$  captures >95% of task-relevant information

**Connection to M4.3 (PCA):** Just as PCA finds that most variance lives in a few principal components, LoRA exploits the fact that weight updates are approximately low-rank.

## LORA: FORWARD PASS

During inference, the output for input  $x$  is:

$$h = W_0x + BAx = (W_0 + BA)x$$

At deployment, merge:  $W_{\text{merged}} = W_0 + BA$  — zero inference overhead

### KEY INSIGHT

$A$  is initialised with Kaiming normal;  $B$  is initialised to zero. So at the start of training,  $\Delta W = BA = 0$  — the model begins as the original pre-trained model.

Scaling factor  $\alpha/r$  controls the magnitude of the update.

## QLORA: 4-BIT TRAINING

QLoRA (Dettmers et al., 2023) makes LoRA even more memory-efficient:

1. **NF4 quantization** — 4-bit NormalFloat: quantization levels optimally spaced for normally distributed weights
2. **Double quantization** — quantize the quantization constants themselves (saves ~0.37 bits/param)
3. **Paged optimizers** — offload optimizer states to CPU when GPU memory spikes

$$Y = X \cdot \text{dequant}(W_{\text{NF4}}) + X \cdot B \cdot A$$

Base weights in 4-bit; LoRA adapters in 16-bit (BFloat16)

Result: fine-tune a 65B model on a single 48GB GPU.

## QLORA MEMORY COMPARISON

| Method                | 7B Model | 13B Model | 65B Model |
|-----------------------|----------|-----------|-----------|
| Full fine-tune (FP16) | ~120 GB  | ~220 GB   | ~780 GB   |
| LoRA (FP16 base)      | ~28 GB   | ~52 GB    | ~260 GB   |
| QLoRA (NF4 base)      | ~6 GB    | ~10 GB    | ~48 GB    |

**Plain English:** QLoRA turns a multi-GPU data centre job into something you can run on a gaming laptop.

## ADAPTER LIFECYCLE

---



Each adapter is a small file (~10–100 MB). You can:

- Stack multiple adapters on the same base model
- A/B test adapters in production
- Roll back to a previous adapter without redownloading the base

# KAILASH BRIDGE: ALIGNMENTPIPELINE

## THEORY → ENGINE

LoRA / QLoRA / SFT theory → AlignmentPipeline + AdapterRegistry

```
from kailash_align import AlignmentPipeline, AlignmentConfig
from kailash_align import AdapterRegistry

config = AlignmentConfig(
    method="sft",
    lora_rank=16,
    lora_alpha=32,
    quantization="nf4",          # QLoRA
    dataset="my_instructions",
)
pipeline = AlignmentPipeline(config)
result = await pipeline.train()

# Register the adapter
registry = AdapterRegistry()
await registry.register(result.adapter, tags=["v1", "sft"])
```

## 6.2 — PREFERENCE ALIGNMENT

---

### WHAT IS ALIGNMENT?

A model that predicts the next word well is not necessarily *helpful*, *harmless*, or *honest*.

**Alignment** = making AI do what humans actually want, not just what they literally said.

**Analogy:** A perfectly obedient employee who follows every instruction literally — "delete all the old files" — is dangerous. You want someone who understands *intent*.

## THE RLHF PIPELINE

---

The original approach (InstructGPT, 2022):

1. SFT → 2. Reward Model → 3. PPO

1. **SFT** — Fine-tune on demonstration data (lesson 6.1)
2. **Reward model** — Train a separate model to score outputs from human preferences
3. **PPO** — Use RL to maximise the reward model's score

**Problem:** Step 2 + 3 require training TWO extra models, are unstable, and sensitive to hyperparameters. Can we do better?

## DPO: SKIP THE REWARD MODEL

---

Direct Preference Optimisation (Rafailov et al., 2023) collapses steps 2 and 3 into a **single training step**:

### RLHF (3 STEPS)

SFT → Train reward model → PPO

3 models in memory, unstable

### DPO (1 STEP)

SFT → DPO on preference pairs

1 model + reference, stable

All you need: pairs of  $(x, y_w, y_l)$  — a prompt, a preferred response, and a rejected response.

## DPO DERIVATION: STEP 1 — BRADLEY-TERRY MODEL

---

Human preferences are modelled with the **Bradley-Terry** model. Given a prompt  $x$  and two responses  $y_1, y_2$ :

$$p^*(y_1 \succ y_2 \mid x) = \sigma(r^*(x, y_1) - r^*(x, y_2))$$

where  $\sigma$  is the sigmoid function and  $r^*$  is the "true" reward

The probability that a human prefers  $y_1$  over  $y_2$  depends on the **difference in rewards**.

## DPO DERIVATION: STEP 2 — THE RLHF OBJECTIVE

Standard RLHF optimises the policy  $\pi_\theta$  to maximise reward while staying close to a reference policy  $\pi_{\text{ref}}$ :

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x)} \left[ r(x, y) - \beta \text{KL}(\pi_\theta(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x)) \right]$$

$\beta$  controls how far the policy can drift from the reference

The KL penalty prevents the model from "gaming" the reward model by producing degenerate outputs.

## DPO DERIVATION: STEP 3 — OPTIMAL POLICY

The KL-constrained objective has a **closed-form optimal policy**:

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y | x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

where  $Z(x) = \sum_y \pi_{\text{ref}}(y|x) \exp(r(x, y)/\beta)$  is the partition function

### HOW?

Take the functional derivative of the objective w.r.t.  $\pi(y|x)$ , set to zero, and solve — this is a standard calculus of variations result for KL-regularised optimisation.

## DPO DERIVATION: STEP 4 — SOLVE FOR REWARD

Rearrange the optimal policy to express the reward in terms of the policy:

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x)$$

### KEY MOVE

Take log of both sides of the optimal policy equation:

$$\log \pi^*(y|x) = \log \pi_{\text{ref}}(y|x) + \frac{1}{\beta} r(x, y) - \log Z(x)$$

Then solve for  $r(x, y)$ .

## DPO DERIVATION: STEP 5 — SUBSTITUTE INTO BRADLEY-TERRY

Substitute the reward expression into the Bradley-Terry preference model:

$$p^*(y_w \succ y_l \mid x) = \sigma \left( \beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right)$$

The  $\beta \log Z(x)$  terms cancel!

### WHY THIS MATTERS

The partition function  $Z(x)$  — which is intractable to compute — drops out. We now have preferences expressed entirely in terms of policy log-ratios.

## DPO DERIVATION: STEP 6 — THE DPO LOSS

---

Maximise the log-likelihood of observed preferences under the policy  $\pi_\theta$ :

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l)} \left[ \log \sigma \left( \beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

This is a **simple binary cross-entropy loss** on log-probability ratios. No reward model. No RL loop. Just supervised learning on preference pairs.

## DPO: WHAT IT MEANS

---

The DPO loss says:

- **Increase** the probability of the preferred response  $y_w$
- **Decrease** the probability of the rejected response  $y_l$
- **Relative to** how the reference model would score them

**Plain English:** "Make the model more likely to say things humans prefer, and less likely to say things humans dislike — but don't drift too far from what the model originally knew."

## GRPO: GROUP RELATIVE POLICY OPTIMISATION

DeepSeek-R1's approach — no reward model, no reference model:

$$\mathcal{L}_{\text{GRPO}} = -\frac{1}{G} \sum_{i=1}^G \frac{1}{|O_i|} \sum_{t=1}^{|O_i|} \hat{A}_i \log \pi_{\theta}(o_t^i \mid q, o_{<t}^i)$$

$$\hat{A}_i = \frac{r_i - \text{mean}(\{r_j\})}{\text{std}(\{r_j\})} \text{ — advantage normalised within the group}$$

- Generate  $G$  outputs per prompt
- Score each with a simple reward (e.g., correctness)
- Normalise rewards within the group → no baseline model needed

## BEYOND DPO

| Method | Key Innovation                                   | When to Use                            |
|--------|--------------------------------------------------|----------------------------------------|
| ORPO   | Combines SFT + preference in one loss            | Limited compute, want single-stage     |
| SimPO  | Length-normalised, no reference model            | When reference model is unavailable    |
| KTO    | Works with unpaired preferences (thumbs up/down) | When you have ratings, not comparisons |
| IPO    | Avoids overfitting to deterministic preferences  | Noisy human labels                     |

**Trend:** The field is moving away from paired preferences (expensive to collect) toward unpaired signals and self-play.

# LLM EVALUATION

| Metric       | What It Measures                                                     | Pitfall                                         |
|--------------|----------------------------------------------------------------------|-------------------------------------------------|
| Perplexity   | $2^{-\frac{1}{N} \sum \log_2 p(w_i)}$ — how "surprised" the model is | Low perplexity $\neq$ good generation           |
| BLEU         | n-gram precision vs reference                                        | Penalises valid paraphrases                     |
| ROUGE        | n-gram recall vs reference                                           | Misses semantic quality                         |
| BERTScore    | Contextual embedding similarity                                      | Expensive, model-dependent                      |
| LLM-as-Judge | Another LLM rates the output                                         | Position bias, verbosity bias, self-enhancement |

Always use multiple metrics. No single number captures "quality".

## LLM-AS-JUDGE: KNOWN BIASES

---

- **Position bias** — tends to prefer the first (or last) option presented
- **Verbosity bias** — longer answers rated higher regardless of quality
- **Self-enhancement** — rates its own outputs higher than other models'
- **Authority bias** — confident-sounding wrong answers beat uncertain correct ones

**Mitigation:** Randomise position, use concise rubrics, employ diverse judge models, and always validate against human agreement rates.

# KAILASH BRIDGE: DPO & EVALUATION

## THEORY → ENGINE

DPO / GRPO / Evaluation theory → AlignmentPipeline(method="dpo")

```
from kailash_align import AlignmentPipeline, AlignmentConfig

config = AlignmentConfig(
    method="dpo",
    beta=0.1,           # KL penalty strength
    lora_rank=16,
    dataset="preference_pairs",
)
pipeline = AlignmentPipeline(config)
result = await pipeline.train()

# Evaluate
from kailash_align import evaluator
scores = await evaluator.evaluate(
    model=result.model,
    metrics=["perplexity", "bertscore", "llm_judge"],
)
```

## 6.3 — REINFORCEMENT LEARNING

---

### WHAT IS RL?

An agent interacts with an environment. It takes **actions**, observes **states**, and receives **rewards**. It learns to maximise cumulative reward through trial and error.

**Analogy:** Teaching a dog new tricks. You don't tell the dog HOW to sit — you reward it when it does. Over many tries, it learns the action-to-reward connection.

## RL VOCABULARY

| Term     | Symbol     | Plain English                                   |
|----------|------------|-------------------------------------------------|
| State    | $s$        | What the world looks like right now             |
| Action   | $a$        | What the agent does                             |
| Reward   | $r$        | How good that action was                        |
| Policy   | $\pi(a s)$ | The agent's strategy: "in this state, do this"  |
| Value    | $V(s)$     | How good is it to BE in this state?             |
| Q-value  | $Q(s, a)$  | How good is it to DO this action in this state? |
| Discount | $\gamma$   | How much we value future vs immediate reward    |

# BELLMAN EXPECTATION EQUATION: DERIVATION

The **value function** is the expected cumulative discounted reward:

## DEFINITION

$$V^\pi(s) = \mathbb{E}_\pi \left[ \sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid S_0 = s \right]$$

## SPLIT FIRST REWARD FROM REST

$$V^\pi(s) = \mathbb{E}_\pi \left[ R_1 + \gamma \sum_{t=0}^{\infty} \gamma^t R_{t+2} \mid S_0 = s \right]$$

## RECOGNISE RECURSION

The inner sum is just  $V^\pi(S_1)$  by the Markov property.

## BELLMAN EXPECTATION: RESULT

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} p(s'|s, a) \left[ r(s, a, s') + \gamma V^\pi(s') \right]$$

### Read it aloud:

- For each action I might take (weighted by my policy)...
- For each state I might land in (weighted by transition probability)...
- Add the immediate reward PLUS the discounted value of where I end up

**This is a system of linear equations** — one per state. For small state spaces, solve directly. For large spaces, approximate.

# BELLMAN OPTIMALITY EQUATION: DERIVATION

The **optimal** value function selects the best action at every step:

## DEFINITION

$$V^*(s) = \max_{\pi} V^{\pi}(s)$$

## OPTIMALITY PRINCIPLE

The optimal policy at state  $s$  takes the action that maximises immediate reward + discounted future value:

$$V^*(s) = \max_a \sum_{s'} p(s'|s, a) \left[ r(s, a, s') + \gamma V^*(s') \right]$$

Replace  $\sum_a \pi(a|s)$  with  $\max_a$  — the optimal policy is deterministic (in finite MDPs).

## BELLMAN FOR Q-VALUES

### EXPECTATION

$$Q^{\pi}(s, a) = \sum_{s'} p(s'|s, a) \left[ r + \gamma \sum_{a'} \pi(a'|s') Q^{\pi}(s', a') \right]$$

### OPTIMALITY

$$Q^*(s, a) = \sum_{s'} p(s'|s, a) \left[ r + \gamma \max_{a'} Q^*(s', a') \right]$$

**Relationship:**  $V^*(s) = \max_a Q^*(s, a)$  and  $\pi^*(s) = \arg \max_a Q^*(s, a)$

## POLICY GRADIENT: THE FOUNDATION OF PPO

---

Instead of learning values, directly optimise the policy parameters  $\theta$ :

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot A^{\pi_{\theta}}(s, a)]$$

The **policy gradient theorem**.  $A(s, a)$  is the advantage: how much better is action  $a$  than average?

**Intuition:** increase the probability of actions that are better than average; decrease those that are worse.

## PPO: CLIPPED OBJECTIVE — STEP 1

---

Define the probability ratio between new and old policy:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

The naive objective would be  $L^{CPI} = \mathbb{E}[r_t(\theta)\hat{A}_t]$ , but this allows **unbounded policy updates** — the policy can change drastically in a single step, causing instability.

**Problem:** TRPO solved this with constrained optimisation (expensive). PPO finds a simpler way.

## PPO: CLIPPED OBJECTIVE — STEP 2

Clip the ratio to prevent large updates:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[ \min \left( r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t \right) \right]$$

### HOW CLIPPING WORKS

- If  $\hat{A}_t > 0$  (good action): ratio capped at  $1 + \epsilon$  — prevents over-exploitation
- If  $\hat{A}_t < 0$  (bad action): ratio floored at  $1 - \epsilon$  — prevents over-avoidance
- Typical  $\epsilon = 0.2$  — policy can change by at most 20% per step

## GAE: GENERALISED ADVANTAGE ESTIMATION

PPO needs advantage estimates  $\hat{A}_t$ . GAE (Schulman et al., 2015) balances bias and variance:

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

where  $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$  is the TD error

- $\lambda = 0$ : one-step TD (low variance, high bias)
- $\lambda = 1$ : Monte Carlo (high variance, low bias)
- $\lambda = 0.95$ : practical sweet spot

## SAC: MAXIMUM ENTROPY RL

Soft Actor-Critic adds an **entropy bonus** to the reward:

$$J(\pi) = \sum_t \mathbb{E}[r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))]$$

$\alpha$  is the temperature — how much to value exploration

- High  $\alpha$ : explore more (spread probability across actions)
- Low  $\alpha$ : exploit more (concentrate on best action)
- $\alpha$  can be learned automatically to target a desired entropy level

**Why entropy?** It prevents premature convergence, improves robustness, and results in policies that can be fine-tuned more easily.

## RLHF AS RL APPLICATION

RLHF connects the alignment block to the RL block:

| RL Concept  | RLHF Mapping                            |
|-------------|-----------------------------------------|
| Environment | The prompt distribution                 |
| State       | Prompt + tokens generated so far        |
| Action      | Next token to generate                  |
| Reward      | Reward model score (at end of sequence) |
| Policy      | The language model $\pi_\theta$         |

**DPO bypasses this:** DPO solves the RL objective in closed form, so you never need to run the RL loop. But understanding the RL formulation explains *why* DPO works.

## OFFLINE RL

---

Learn a policy from a fixed dataset — no further environment interaction.

### CONSERVATIVE Q-LEARNING (CQL)

Penalises Q-values for unseen actions, preventing overestimation:

$$\hat{Q} \leq Q^{\pi_{\beta}} \text{ (conservative)}$$

### DECISION TRANSFORMER

Frames RL as sequence modelling: condition on desired return, predict actions.

No Bellman equation needed — just a transformer.

**Use case:** Healthcare, finance — anywhere you cannot experiment in the real environment.

# KAILASH BRIDGE: RLTRAINER

## THEORY → ENGINE

Bellman / PPO / SAC theory → RLTrainer (backed by Stable-Baselines3)

```
from kailash_ml.rl import RLTrainer, env_registry, policy_registry

# Register a custom Gymnasium environment
env_registry.register("portfolio_env", PortfolioEnv)

trainer = RLTrainer(
    algorithm="ppo",          # or "sac", "dqn"
    env_name="portfolio_env",
    policy="MlpPolicy",
    total_timesteps=100_000,
    clip_range=0.2,           # PPO epsilon
    gae_lambda=0.95,
)
model = await trainer.train()
reward = await trainer.evaluate(model, n_episodes=50)
```

## 6.4 — ADVANCED ALIGNMENT: MODEL MERGING

---

### WHAT IF YOU COULD COMBINE ADAPTERS WITHOUT RETRAINING?

Train separate LoRA adapters for different skills, then merge them into a single model:



## MERGING METHODS

| Method             | Mechanism                             | Strength                                     |
|--------------------|---------------------------------------|----------------------------------------------|
| <b>Model Soups</b> | Weighted average of parameters        | Simple, effective for similar models         |
| <b>TIES</b>        | Trim low-magnitude, Elect sign, Merge | Handles conflicting parameter signs          |
| <b>DARE</b>        | Randomly drop delta params, rescale   | Reduces interference between tasks           |
| <b>SLERP</b>       | Spherical linear interpolation        | Smooth blending on the parameter hypersphere |

**Why merge?** Training one adapter per task is cheaper than training one model for all tasks. Merging is a post-hoc combination strategy.

## TIES MERGING: STEP BY STEP

1. **Trim:** Zero out parameters with small magnitude changes (below a threshold  $\lambda$ )
2. **Elect sign:** For each parameter position, choose the sign that appears most frequently across models
3. **Merge:** Average only the parameters that agree with the elected sign

$$\tau_m = \text{sign}\left(\sum_k \tilde{\tau}_k^m\right) \cdot \frac{\sum_k |\tilde{\tau}_k^m| \cdot 1[\text{sign}(\tilde{\tau}_k^m) = \text{elected}]}{|\{k : \text{sign}(\tilde{\tau}_k^m) = \text{elected}\}|}$$

Conflicting updates cancel; agreeing updates reinforce

## DIFFERENTIAL PRIVACY IN ALIGNMENT

When fine-tuning on sensitive data, DP-SGD provides formal privacy guarantees:

$$\tilde{g}_t = \frac{1}{B} \sum_{i=1}^B \text{clip}(g_i, C) + \mathcal{N}(0, \sigma^2 C^2 I)$$

Clip gradients to norm  $C$ , add calibrated Gaussian noise

- Privacy budget  $(\epsilon, \delta)$ : lower  $\epsilon$  = stronger privacy
- Trade-off: more noise  $\rightarrow$  worse model quality
- LoRA + DP-SGD: fewer parameters  $\rightarrow$  less noise needed  $\rightarrow$  better privacy-utility trade-off

# KAILASH BRIDGE: MERGING & EVALUATION

## THEORY → ENGINE

TIES / DARE / Model Soups → `kailash_align.merge`

```
from kailash_align import merge, evaluator, AdapterRegistry

registry = AdapterRegistry()
adapter_sft = await registry.get("sft-v1")
adapter_dpo = await registry.get("dpo-v1")

# TIES merge
merged = await merge(
    adapters=[adapter_sft, adapter_dpo],
    method="ties",
    trim_threshold=0.1,
)

# Compare all three
results = await evaluator.compare(
    models=[adapter_sft.model, adapter_dpo.model, merged],
    metrics=["perplexity", "bertscore", "llm_judge"],
)
```

## 6.5 — AI GOVERNANCE WITH PACT

---

### WHAT IS GOVERNANCE?

Rules about **who can do what**, enforced **automatically**.

**Analogy:** A hotel key card. You don't ask the front desk every time you enter your room — the card encodes your access rights, and the lock enforces them. PACT is the key card system for AI agents.

Without governance, every AI system is a skeleton key.

## EU AI ACT: RISK TIERS

The EU AI Act classifies AI systems by risk level:

| Tier         | Examples                                              | Requirements                                           |
|--------------|-------------------------------------------------------|--------------------------------------------------------|
| Unacceptable | Social scoring, real-time biometric mass surveillance | Banned (Art. 5)                                        |
| High         | Credit scoring, recruitment, medical devices          | Conformity assessment, human oversight (Art. 6, 9, 13) |
| Limited      | Chatbots, deepfakes, emotion recognition              | Transparency obligations (Art. 52)                     |
| Minimal      | Spam filters, game AI                                 | No requirements (voluntary codes of conduct)           |

## KEY EU AI ACT ARTICLES

---

- **Article 6** — Classification rules for high-risk AI (Annex III list + safety component test)
- **Article 9** — Risk management system: identify, analyse, evaluate, mitigate. Continuous. Documented.
- **Article 13** — Transparency and provision of information: "sufficient for the user to interpret the system's output"
- **Article 52** — Disclosure obligations: users must know they are interacting with AI

**Fines:** Up to 7% of global annual turnover for prohibited practices. Up to 3% for other non-compliance.

## SINGAPORE: AI VERIFY & ISAGO 2.0

---

Singapore's approach: voluntary but market-expected

- **AI Verify** — open-source testing toolkit for responsible AI
- **ISAGO 2.0** — International Standard for AI Governance and Operations
- Covers: accountability, data management, performance, safety, fairness, transparency
- Testing components: fairness metrics, explainability reports, robustness checks

**Singapore context:** MAS (Monetary Authority of Singapore) already requires AI governance for financial institutions. ISAGO 2.0 gives a structured framework for compliance.

## PACT: THE GOVERNANCE GRAMMAR

---

PACT provides a **formal grammar** for organisational governance. Everything reduces to three letters:

**D**

**Department**

The organisational unit

**T**

**Team**

The functional group

**R**

**Role**

The individual capability

Every entity in the system — human or AI — has a D/T/R address.

## D/T/R ADDRESSING

---

Addresses are hierarchical paths:

```
org/ ├── engineering/ ← D: Department | ├── ml-platform/ ← T: Team | | ├── senior-ml-engineer ← R: Role |  
    | ├── junior-ml-engineer ← R: Role | | └── ml-agent-alpha ← R: Role (AI) | └── backend/ | ├── senior-  
    backend-dev | └── api-agent-beta ← R: Role (AI) └── compliance/ └── governance/ └── compliance-officer
```

Address format: `org/engineering/ml-platform/senior-ml-engineer`

## MONOTONIC TIGHTENING

Permissions can only get **more restrictive** as you go deeper:

$$\text{envelope}(R) \subseteq \text{envelope}(T) \subseteq \text{envelope}(D)$$

A role can never have MORE permissions than its team, which can never exceed its department

- If Engineering can access datasets A, B, C — ML Platform can access at most A, B, C
- If ML Platform can deploy to staging — junior-ml-engineer can deploy to at most staging
- Tightening is **monotonic**: once restricted, cannot be un-restricted by a child

**Fail-closed:** If any level in the chain denies access, access is denied. No override. No exceptions.

## OPERATING ENVELOPES

Every role has a **role envelope**. Every task has a **task envelope**. The agent can only act in the intersection:

$$\text{effective envelope} = \text{role envelope} \cap \text{task envelope}$$

What you CAN do  $\cap$  what you are ASKED to do = what actually happens

### ROLE ENVELOPE

- Max cost per request: \$5
- Allowed tools: DataExplorer, polars
- Data access: public datasets only

### TASK ENVELOPE

- Budget: \$2
- Required tools: DataExplorer
- Data: HDB prices dataset

# KNOWLEDGE CLEARANCE

Classification of data sensitivity — independent of authority:

| Level | Name         | Example               | Who Can Access       |
|-------|--------------|-----------------------|----------------------|
| 1     | Public       | Marketing materials   | Any agent            |
| 2     | Internal     | Company wiki          | Authenticated agents |
| 3     | Confidential | Customer data         | Team-level clearance |
| 4     | Restricted   | Financial projections | Named roles only     |
| 5     | Critical     | Encryption keys, PII  | Human-approved only  |

An agent with Level 3 clearance cannot access Level 4 data, even if its role envelope technically allows the operation.

## ENFORCEMENT: TRUSTPOSTURE & CONFIDENTIALITYLEVEL

---

### TRUSTPOSTURE

- **Untrusted** — deny all by default
- **Cautious** — allow reads, deny writes
- **Standard** — normal operations
- **Elevated** — expanded access for senior roles

### CONFIDENTIALITYLEVEL

Maps to knowledge clearance 1–5. Attached to every piece of data, every tool, every endpoint.

The **trust plane** underneath everything: TrustPosture determines what you can do; ConfidentialityLevel determines what you can see.

# KAILASH BRIDGE: GOVERNANCEENGINE

## THEORY → ENGINE

PACT D/T/R + Operating Envelopes → GovernanceEngine + compile\_org()

```
from kailash_pact import GovernanceEngine, Address

engine = GovernanceEngine()
org = engine.compile_org({
    "departments": [{
        "name": "engineering",
        "teams": [{
            "name": "ml-platform",
            "roles": ["senior-ml-eng", "ml-agent"],
            "max_cost_per_request": 10.0,
            "allowed_tools": ["DataExplorer", "TrainingPipeline"],
        }]
    }]
})

addr = Address("org/engineering/ml-platform/ml-agent")
can = engine.can_access(addr, resource="customer_data", level=3)
reason = engine.explain_access(addr, resource="customer_data")
```

## 6.6 — GOVERNED AGENTS

---

An agent without governance is a liability. A governed agent is an asset.

### UNGOVERNED AGENT

- Can call any API
- No spending limit
- Accesses all data
- No audit trail

### PACT-GOVERNED AGENT

- Tools restricted by role
- Budget enforced per-task
- Data filtered by clearance
- Every action logged

## GOVERNANCECONTEXT: FROZEN BY DESIGN

---

When a governed agent starts a task, it receives a **frozen GovernanceContext**:

- The context is computed at task start from the D/T/R address + task envelope
- It is **immutable** — the agent cannot modify its own permissions mid-task
- Every tool call checks the context before execution
- If the context denies access, the operation fails immediately (fail-closed)

**Why frozen?** If an agent could modify its own context, a prompt injection could escalate privileges. Frozen contexts are immune to this attack.

# ROLENVELOPE & TASKENVELOPE

## ROLENVELOPE

```
{
  "max_cost": 50.0,
  "allowed_tools": [
    "DataExplorer",
    "TrainingPipeline"
  ],
  "clearance": 3,
  "can_deploy": False,
  "can_promote": False,
}
```

## TASKENVELOPE

```
{
  "budget": 10.0,
  "required_tools": [
    "DataExplorer"
  ],
  "data_scope": "hdb_prices",
  "timeout_seconds": 300,
  "max_retries": 3,
}
```

Effective:  $\text{max\_cost} = \min(50, 10) = \$10$ . Tools = intersection. Clearance =  $\min(3, \text{task default})$ .

## AUDIT CHAINS

---

Every governed action produces an audit record:

```
{
  "timestamp": "2025-03-15T09:23:41Z",
  "agent": "org/engineering/ml-platform/ml-agent-alpha",
  "action": "TrainingPipeline.train",
  "decision": "ALLOWED",
  "context": {
    "role_envelope": {"max_cost": 50.0, "clearance": 3},
    "task_envelope": {"budget": 10.0},
    "effective_budget": 10.0
  },
  "cost_incurred": 2.34,
  "budget_remaining": 7.66
}
```

When the regulator asks "why did it decide this?", you hand them the audit chain.

# KAILASH BRIDGE: PACTGOVERNEDAGENT

## THEORY → ENGINE

Frozen context + envelopes + audit → PactGovernedAgent

```
from kailash_pact import PactGovernedAgent, GovernanceContext
from kailash_pact import RoleEnvelope, TaskEnvelope

role = RoleEnvelope(max_cost=50.0, clearance=3,
                    allowed_tools=["DataExplorer", "TrainingPipeline"])
task = TaskEnvelope(budget=10.0, data_scope="hdb_prices")
ctx = GovernanceContext(address=addr, role=role, task=task)

agent = PactGovernedAgent(
    base_agent=my_react_agent,
    governance_context=ctx,    # Frozen at creation
)
result = await agent.run("Analyse HDB price trends")
```

## 6.7 — AGENT GOVERNANCE AT SCALE

---

One governed agent is straightforward. What about **fifty agents** working together?

- Budgets must cascade: team budget → individual agent budgets
- Clearance must propagate: an agent cannot delegate to a higher-clearance agent
- Failures must be handled: what happens when an agent exceeds its budget mid-task?

## BUDGET CASCADING

$$\sum_{i=1}^n b_i \leq B_{\text{team}} \leq B_{\text{department}}$$

Individual budgets must sum to at most the team budget, which is bounded by the department budget

- A supervisor agent allocates budget to worker agents
- Allocation is **pre-committed**: the budget is reserved before the task starts
- Unspent budget returns to the pool after task completion
- Over-budget triggers immediate halt (not a warning — a hard stop)

## CLEARANCE PROPAGATION

---

When Agent A delegates to Agent B:

$$\text{clearance}(B_{\text{effective}}) = \min(\text{clearance}(B_{\text{role}}), \text{clearance}(A_{\text{task}}))$$

The delegated agent gets the *lesser* of its own clearance and the delegating task's clearance

**Anti-pattern:** Agent A (clearance 2) delegates to Agent B (clearance 4) to access restricted data. PACT prevents this: B's effective clearance becomes  $\min(4, 2) = 2$ .

## DERELICTION: WHEN AGENTS FAIL

| Failure Mode        | What Happens                               | PACT Response                               |
|---------------------|--------------------------------------------|---------------------------------------------|
| Budget exceeded     | Agent tries to spend beyond limit          | Hard stop, audit log, supervisor notified   |
| Clearance violation | Agent tries to access restricted data      | Immediate denial, incident logged           |
| Timeout             | Agent runs beyond time envelope            | Graceful shutdown, partial results returned |
| Tool misuse         | Agent calls a tool outside its allowed set | Call blocked, agent receives error          |
| Cascade failure     | Worker agent fails, supervisor must decide | Supervisor can retry, reassign, or escalate |

# ALGORITHMIC IMPACT ASSESSMENT

---

Before deploying a governed agent system, conduct an AIA:

1. **Scope** — What decisions does the system make or influence?
2. **Stakeholders** — Who is affected? (customers, employees, public)
3. **Risk analysis** — What could go wrong? (bias, privacy, safety, economic)
4. **Mitigation** — What controls are in place? (PACT envelopes, audit, human review)
5. **Monitoring** — How will you detect problems post-deployment? (DriftMonitor, audit alerts)

**EU AI Act Art. 9** requires this for high-risk systems. PACT's audit chain provides the evidence base.

# KAILASH BRIDGE: GOVERNANCE AT SCALE

## THEORY → ENGINE

Budget cascading + clearance + dereliction → `kaizen_agents.governance`

```
from kaizen_agents.governance import (
    ClearanceLevel, BudgetCascade, GovernanceTestHarness
)

# Budget cascade
cascade = BudgetCascade(
    team_budget=100.0,
    allocations={
        "data-agent": 30.0,
        "training-agent": 50.0,
        "eval-agent": 20.0,
    }
)

# Governance testing
harness = GovernanceTestHarness(org=compiled_org)
report = await harness.test_clearance_propagation()
report = await harness.test_budget_enforcement()
report = await harness.test_dereliction_handling()
```

## 6.8 — CAPSTONE: FULL PLATFORM

---

The capstone integrates all 8 Kailash packages into a single governed system:



Your task: build a governed ML pipeline where agents explore data, train models, evaluate results, and deploy — all within PACT operating envelopes.

Scaffolding: ~40% (integration-focused, not from-scratch)

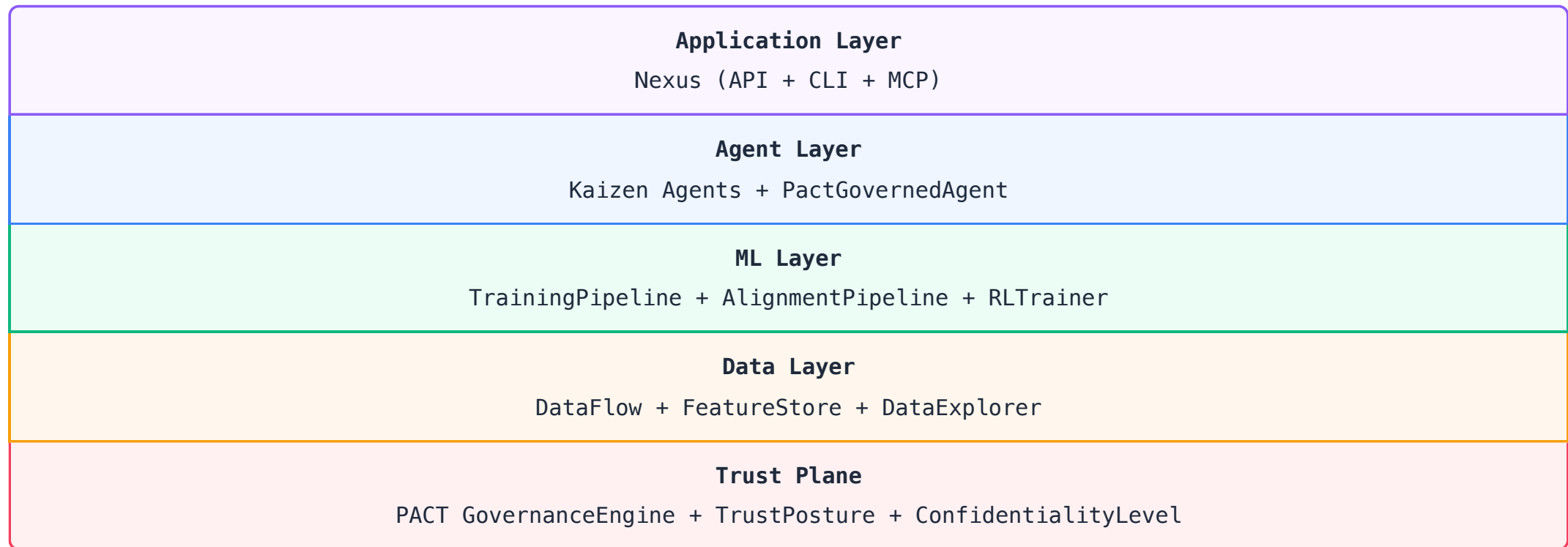
## KAILASH ENGINE MAP: MODULE 6

| Engine            | Package       | Purpose                                          |
|-------------------|---------------|--------------------------------------------------|
| AlignmentPipeline | kailash-align | SFT, DPO, QLoRA — one unified API                |
| AdapterRegistry   | kailash-align | Adapter lifecycle: train → eval → merge → deploy |
| GovernanceEngine  | kailash-pact  | Define org, compile rules, enforce access        |
| PactGovernedAgent | kailash-pact  | Wrap any agent with governance                   |
| RLTrainer         | kailash-ml    | PPO, SAC, DQN — Stable-Baselines3 backend        |
| Nexus             | kailash-nexus | API + CLI + MCP deployment                       |

## THEORY-TO-ENGINE MAPPING

| Mathematical Concept            | Kailash API                                              |
|---------------------------------|----------------------------------------------------------|
| LoRA: $W = W_0 + BA$            | <code>AlignmentConfig(method="sft", lora_rank=16)</code> |
| QLoRA: NF4 quantization         | <code>AlignmentConfig(quantization="nf4")</code>         |
| DPO: $\mathcal{L}_{\text{DPO}}$ | <code>AlignmentConfig(method="dpo", beta=0.1)</code>     |
| TIES merging                    | <code>merge(adapters, method="ties")</code>              |
| PPO: $L^{\text{CLIP}}$          | <code>RLTrainer(algorithm="ppo", clip_range=0.2)</code>  |
| SAC: max-entropy RL             | <code>RLTrainer(algorithm="sac")</code>                  |
| D/T/R addressing                | <code>Address("org/dept/team/role")</code>               |
| Operating envelopes             | <code>RoleEnvelope</code> n <code>TaskEnvelope</code>    |

# ARCHITECTURE: THE TRUST STACK



The trust plane is *underneath* everything — every layer checks it before acting.

# LAB SETUP

## EXERCISE PROGRESSION

| Exercise | Topic                  | What You Build                      |
|----------|------------------------|-------------------------------------|
| ex_1     | SFT Fine-Tuning        | LoRA adapter with AlignmentPipeline |
| ex_2     | Preference Alignment   | DPO training + evaluation           |
| ex_3     | AI Governance          | GovernanceEngine org with D/T/R     |
| ex_4     | Governed Agents        | PactGovernedAgent with envelopes    |
| ex_5     | Reinforcement Learning | PPO agent in custom environment     |
| ex_6     | Full Platform Capstone | All 8 packages integrated           |

Open: `modules/pcml6/local/ex_1.py`

## ENVIRONMENT SETUP

---

```
from dotenv import load_dotenv
load_dotenv()

from shared.data_loader import PCMLDataLoader
loader = PCMLDataLoader()

# Module 6 packages
# pip install kailash-align kailash-pact kailash-ml kailash-nexus
```

**GPU note:** SFT/DPO exercises work with QLoRA on a single GPU (or Colab T4). RL exercises run on CPU. Governance exercises need no GPU.

## DATA LOADING

```
# All formats use the same loader
from shared.data_loader import PCMLDataLoader
import polars as pl

loader = PCMLDataLoader()

# Preference data for DPO
prefs = loader.load("pcml6", "preference_pairs.jsonl")

# RL environment config
env_config = loader.load("pcml6", "env_config.yaml")

# Governance org definition
org_spec = loader.load("pcml6", "org_structure.yaml")
```

Never hardcode paths. Never use pandas.

## DISCUSSION: ALIGNMENT CHOICES

---

1. "Your company has 1,000 customer service transcripts rated 1-5 stars. Should you use SFT, DPO, or both? What data preparation is needed for each?"
2. "A fine-tuned model scores well on benchmarks but users complain it's 'too formal'. What went wrong? How would you fix it?"
3. "You merge two adapters (code + chat) and the merged model is worse than either alone. What merging method would you try next, and why?"

## DISCUSSION: GOVERNANCE SCENARIOS

---

1. "Your ML agent discovers a data quality issue at 2am and wants to retrain the model. Its role envelope allows training but requires human approval for production deployment. What happens?"
2. "A junior analyst asks: 'Why can't my agent access the financial forecasts? The agent is smarter than me.' How do you explain knowledge clearance?"
3. "The board asks: 'How do we know our AI agents aren't accessing data they shouldn't?' Walk them through the PACT audit chain."

## DISCUSSION: RL APPLICATIONS

---

1. "You want to optimise a portfolio rebalancing strategy. Why might offline RL (CQL) be preferable to online PPO?"
2. "Your PPO agent's reward suddenly spikes then collapses during training. What diagnostic would you run first?"
3. "DPO and PPO both produce aligned models. When would you choose the RL-based approach (PPO + reward model) over DPO?"

## KEY TAKEAWAYS: EVERYONE

---

- **Fine-tuning** teaches a pre-trained model your domain; LoRA makes it affordable
- **Alignment** makes AI do what you want, not just what you said
- **Governance** is automated enforcement: D/T/R addresses, operating envelopes, knowledge clearance
- **RL** learns from trial and error — reward signals, not labelled data
- Every concept maps to a Kailash engine with a production-ready API

## KEY TAKEAWAYS: IF YOU FOLLOWED THE MATH

---

- **LoRA** works because weight updates are intrinsically low-rank:  $\Delta W = BA$  with  $r \ll d$
- **DPO** derives from Bradley-Terry + KL-constrained RLHF  $\rightarrow$  closed-form loss on preference pairs
- **Bellman** equations (expectation and optimality) are the foundation of all value-based RL
- **PPO** clips the policy ratio to prevent destructive updates:  $[1 - \epsilon, 1 + \epsilon]$
- **PACT envelopes** are formally  $\text{role} \cap \text{task}$  with monotonic tightening

## KEY TAKEAWAYS: FOR THE EXPERTS

---

- **ORPO/SimPO/KTO** eliminate the need for paired preferences or reference models
- **TIES/DARE** merging handles parameter conflicts via sign election and random dropout
- **Offline RL** (CQL, Decision Transformer) enables learning without live environments
- **DP-SGD** provides formal privacy guarantees for alignment training
- **Algorithmic impact assessments** are the bridge between technical governance and regulatory compliance

## CUMULATIVE KAILASH ENGINE MAP

| Module    | Engines Unlocked                                                                          |
|-----------|-------------------------------------------------------------------------------------------|
| M1        | DataExplorer, PreprocessingPipeline, ModelVisualizer                                      |
| M2        | FeatureStore, FeatureEngineer, ExperimentTracker                                          |
| M3        | TrainingPipeline, HyperparameterSearch, ModelRegistry, WorkflowBuilder                    |
| M4        | AutoMLEngine, EnsembleEngine, DriftMonitor, InferenceServer                               |
| M5        | Delegate, ReActAgent, RAGResearchAgent, Nexus                                             |
| <b>M6</b> | <b>AlignmentPipeline, AdapterRegistry, GovernanceEngine, PactGovernedAgent, RLTrainer</b> |

You now have the complete Kailash toolkit. The capstone integrates all of it.

## WHAT COMES NEXT

---

### CAPSTONE (6.8)

- Build a complete governed ML platform
- All 8 Kailash packages integrated
- ~40% scaffolding (integration, not from-scratch)
- Debugging traces, testing agents

### AFTER PCML

- Kailash SDK documentation: `terrene.dev`
- Contribute to the open-source ecosystem
- Build production systems with governance from day one
- Advanced Certificate awarded (M5+M6)

## ASSESSMENT PREVIEW

---

### QUIZ TOPICS

- LoRA: why low-rank, when to increase rank, adapter lifecycle
- DPO: what the loss optimises, role of  $\beta$ , data requirements
- PACT: D/T/R addressing, monotonic tightening, fail-closed
- RL: value vs policy methods, PPO clipping, reward design
- Governance: EU AI Act risk tiers, AI Verify components, operating envelopes

### CAPSTONE ASSESSMENT

Exercise 6: deploy a governed agent pipeline. Assessed on: correct integration, envelope enforcement, audit completeness.

# MODULE 6 COMPLETE

## ALIGNMENT, GOVERNANCE & ORGANISATIONAL TRANSFORMATION

Professional Certificate in Machine Learning — Terrene Open Academy

*"Build AI that does what you want, controlled by who you trust."*